



2D Videohra Salvation Path: Uživatelské rozhraní

2D Video game Salvation Path: User interface

Maturitní práce

Vedoucí práce: Bc. Adam Tyroň

2025/26

Jonáš Kymel, 4.D

Abstrakt

Tato práce se zabývá vývojem videohry Surreal Bowling: Salvation Path a patří do skupiny tří maturitních prací ve třídě 4.D, které jsou na tuto videohru zaměřeny. Náš nezávislý tým se jmenuje Still Thinking Studio. Všechny tři maturitní práce řeší z většiny programovací a logickou část toho, jak videohra funguje, avšak má práce se zaměřuje i na audiovizuální stránku této videohry, jelikož mám na starosti veškerý audio design, vizuální efekty a post-processing. Co se týče kódové části, věnuji se zejména uživatelskému rozhraní a logice, která odpovídá za správný chod událostí a různých managerů.

Klíčová slova

videohra, 2D plošinovka, Unity, C#

Abstract

This thesis examines the development of the video game Surreal Bowling: Salvation Path and is one of three graduation theses in class 4.D that focus on this particular video game. Our independent team is referred to as Still Thinking Studio. The three graduation projects primarily address the programming and logical components of the video game's functionality. However, my own work also focuses on the audiovisual elements of the video game, as I am responsible for all audio design, visual effects, and post-processing. As for the coding part, the primary focus is on the user interface and the logic that is responsible for the proper functioning of events and various managers.

Key words

video game, 2D platformer, Unity, C#

Poděkování

Rád bych poděkoval panu Bc. Adamu Tyronovi za viceletou aktivitu na Discord serveru naseho vyvojarskeho tymu a ujmuti se nás jako mentor maturitní práce. Dále chci samozřejmě poděkovat svým spolupracovníkům a zároveň spolužákům, s kterými videohry vyvíjím - Danielu Baierovi a Simeonu Bednarzovi za vyvíjení kódu a Jakubu Wojtasovi za téměř veškeré grafické zpracování.

Prohlášení

Prohlašuji, že jsem svou maturitní práci vypracoval/a samostatně. K práci jsem použil/a literaturu a prameny, uvedené v seznamu. Souhlasím s tím, aby moje ročníková práce byla využívána na Střední průmyslové škole Karviná, příspěvkové organizaci.

V Karviné dne: Podpis:

Obsah

Obsah	4
1 Úvod	6
2 Terminologie	7
2.1 GameObject	7
2.2 OST	7
2.3 Post-processing	7
2.4 Unity Editor	7
3 Použitý framework aplikace	8
3.1 Operační systém	8
3.2 Programování	8
3.2.1 C#	8
3.2.2 IDE	8
3.2.3 Version Control	8
3.2.4 Vim	9
3.2.5 AI	9
3.3 Herní engine	9
3.4 Hudba a zvuk	10
3.4.1 FL Studio	10
3.4.2 FMOD Studio	10
3.5 Ostatní	10
4 Celková koncepce řešení	11
4.1 Game Manager	11
4.2 Umírání	11
4.2.1 Checkpoint System	11
4.3 Emotion System	12
4.4 UI	12
4.5 Vizuální efekty	13
4.6 Audio	13
4.6.1 OST	13
4.6.2 SFX	14
4.7 Post-processing	14

5	Popis aplikace	15
5.1	Zpracování základního modelu hry	15
5.1.1	Struktura scén	15
5.1.2	Herní manažery	15
5.1.3	Input systém	17
5.1.4	Základní herní objekty	20
5.2	Vytvoření uživatelského rozhraní	20
5.2.1	UI Toolkit	20
5.2.2	UI Prvky	22
5.2.3	Ostatní funkce UI	25
5.3	Tvorba vizuálních efektů	28
5.3.1	Light 2D	28
5.3.2	Particle System	29
5.3.3	Ostatní efekty	30
5.4	Tvorba zvukových efektů	31
5.4.1	Audio systém	31
5.4.2	Tvorba v FL Studio	40
5.4.3	Kategorizace zvuků	40
5.4.4	Hudba	40
5.5	Post-processing hry	40
6	Závěr	41
	Seznam obrázků	42

1 Úvod

Cílem práce je vyvinout základní komponenty uživatelského rozhraní videohry žánru 2D plošinovka, postarat se o správnou funkčnost game manageru, který se stará například o přepínání mezi jednotlivými scénami, audio manageru, jehož úkolem je přehrávat všechny zvukové efekty a OST ve správný čas správným způsobem, a jednotlivých UI elementů.

Dále se zabývám tvorbou vizuálních efektů, které jsou v herním průmyslu důležité pro zkrášlení grafické stránky a přidání jistého realismu do zážitku z prostředí videohry. Vizuální efekty pomáhají hráčům chápat systémy, rychleji registrovat, co se děje, dostávat pozitivní/negativní zpětnou vazbu a vtažení do děje.

Úzce s vizuálními efekty souvisí i zvukové efekty - snad každý vizuální efekt může mít svůj korespondující zvukový efekt, který ještě více potlačí účinek celé specifické akce. Audio je mezi průměrnými hráči často přehlížené, protože se bere jako samozřejmost vzhledem k tomu že zvukové odezvy se v reálném světě odehrávají všude kolem nás. Zkuste si ale někdy vaši oblíbenou videohru zahrát kompletně bez zvuku - potěšení z videohry v takovém případě u mnoha titulů může klesnout drasticky. Zaměřuji se tedy jak na zvukové efekty, tak i na hudební složku.

Nakonec nastinuji i jednoduchý model postprocessingu, který grafice naší hry dodává atmosféru, aby se tolik nepodobala retro videohrám jako je například herní titul Super Mario Bros.

2 Terminologie

2.1 GameObject

Základní stavební prvek Unity enginu, reprezentující jakýkoli objekt ve scéně. To mohou být například postavy, budovy, zdroje světla, zdroje zvuku, textová pole nebo neviditelné objekty, které slouží čistě pro funkční účely (za pomoci skriptů připojených jako komponent na daný GameObject).

2.2 OST

Zkratka OST (Original Soundtrack) označuje hudební složku vytvořenou přímo pro daný film, seriál, videohru nebo jiné podobné médium.

2.3 Post-processing

Post-processing je proces aplikování vizuálních vylepšení, filtrů a posílení žádoucí atmosféry různými efekty. Mezi často používané efekty patří Color Grading, Bloom nebo Depth of Field.

2.4 Unity Editor

Rozhraní Unity enginu, ve kterém se pracuje většinu času při vývoji videohry (podobně jako kreativní programy Blender, DaVinci Resolve nebo FL Studio).

3 Použitý framework aplikace

3.1 Operační systém

V týmu jako hlavní operační systém pro vývoj Surreal Bowling: Salvation Path používáme operační systém Linux. Já konkrétně používám distribuci Arch Linux, ale v našem týmu můžeme najít i Fedora Linux nebo Ubuntu. Rozhodli jsme se tak, jelikož Linux z většiny používáme jako náš každodenní operační systém a nabízí rychlejší workflow než Windows. Také jsme těžkou cestou zjistili, že pokud na Unity pracují dva vývojáři, jeden z Linuxu a druhý z Windowsu, stává se, že se některé soubory v Unity projektu stanou „corrupted“. Této nepříjemné záležitosti jsme se chtěli vyvarovat a tak jsme se skupinově dohodli právě na Linuxu. Naštěstí vývoj videoher v operačním systému Linux je dnes celkem proveditelný a programy jako Unity Engine, VS Code nebo Git fungují stejně dobře, ne-li lépe, než na Windows.

Jako window manager používám dynamický, dlaždicový Hyprland, který rychlost mé práce zvyšuje mnohonásobně oproti klasickému Windows.

3.2 Programování

3.2.1 C#

Pro psaní kódu jsme použili multiplatformní programovací jazyk C#, mimo jiné i nejpopulárnější jazyk pro .NET. Více o C# v části „Herní engine“.

3.2.2 IDE

Co se týče IDE, osobně přeskakuji mezi různými modifikacemi binárek programu VS Code, takže jsem používal například VSCodium nebo čistý Code OSS, ale momentálně používám klasický VS Code z důvodu kompatibility s nejvíce pluginy třetích stran.

3.2.3 Version Control

Pro spolupráci ve vývojářském týmu používáme Git – systém pro správu verzí, neboli „version control system“ (VCS). Přes Git posíláme lokální změny do online repozitáře našeho projektu na webovou platformu GitHub, kde má přístup i náš mentor práce, aby měl bohatý přehled o tom, jak se nám s vývojem vede.

3.2.4 Vim

Dále, stejně jako Hyprland pro Linux, i pro VS Code používám software, který mi zjednodušuje a urychluje práci – plugin Vim (vscodevim), což je emulátor funkcí „Vim movements“ pro VS Code. Vim je originálně nejznámější modulární textový editor, který je populární díky své rigidní funkci práce s textem a orientaci v něm.

3.2.5 AI

Pro usnadnění psaní kódu, provádění repetitivních úkolů nebo přesouvání větších objemů dat jsem použil AI asistenty ChatGPT a Augment Code. Později jsem zprovoznil vlastní lokální LLM (Large Language Model) v programu LM Studio (konkrétně qwen/qwen3.5-9b) a ten jsem integroval do nástroje OpenCode, což mi umožňuje daný model používat jako agenta v terminálu, který dokáže indexovat celé projekty a provádět v nich úpravy. Díky tomuto řešení tak nejsem omezen žádným předplatným a všechna data zůstávají offline na mém disku.

3.3 Herní engine

Jakožto nejdůležitější software pro vývoj videohry dnes je herní engine. Pro náš případ jsme zvolili multiplatformní herní engine Unity vyvinutý společností Unity Technologies z několika prostých důvodů. Jeden z nejzákladnějších je ten, že v Unity se defaultně programuje v programovacím jazyce C#, přičemž my, jakožto studenti oboru IT na SPŠ Karviná s nástupem v roce 2022, jsme se programování v C# začali učit už v prvním ročníku v předmětu Programování. Díky tomu jsme měli už potřebné znalosti syntaxe a pravidel, ve kterých C# funguje.

Dále jsme také brali v potaz licence různých herních enginů. Mezi nejznámější patří právě Unity, Unreal Engine, Godot Engine nebo GameMaker. Unity nám prozatím přišlo jako nejrozumnější cesta, kde vydávat videohry můžeme pod Unity Free License, pokud jednotlivec nebo tým z výdělků z videohry nebo od sponzora přijímá méně než 200 000 amerických dolarů za posledních 12 měsíců. Jedna z největších nevýhod je, že při startu zadarmo vyvinuté videohry v Unity se při startu videohry objeví úvodní obrazovka (splash screen) s nápisem „Made with Unity“, což našemu týmu momentálně zas tak nevadí.

Naše videohra také není nijak složitá v porovnání s dnešním videoherním standardem a Unity se obecně považuje jako příjemnější volba pro menší projekty, jako mohou být mobilní aplikace nebo právě jednoduchý 2D projekt, jako je naše videohra.

3.4 Hudba a zvuk

3.4.1 FL Studio

Pro tvorbu hudby a zvukových efektů používám osobně vlastněný program FL Studio. Je to můj nejoblíbenější software, ve kterém ze všech zmíněných trávím nejvíce času.

FL Studio je software (Digital Audio Workstation) pro skládání, produkci a mix hudby za pomoci digitálních nástrojů, pluginů, samplů a MIDI signálů.

Mezi pluginy třetích stran, které často používám pro tvorbu či modifikaci zvuku, patří LABS, FLEX, Sytrus, Delay.

3.4.2 FMOD Studio

Původně jsem pro implementaci zvuku do naší hry používal low-level nástroje přímo od Unity, například AudioSource, AudioListener a AudioMixer.

FMOD Studio jsem náhodou objevil během účasti na Brackeys Game Jam 2026.1, kde jsem ihned využil příležitosti tento software využít v praxi. FMOD Studio je profesionální nástroj (audio middleware) pro úpravu a implementaci zvuku do videoher, vyvinutý společností Firelight Technologies. Umožňuje sound designerům vytvářet komplexní zvukové systémy pomocí přívětivého rozhraní bez nutnosti složitého programování, zatímco vývojáři mohou tyto zvuky snadno propojit s kódem pomocí Unity integrace. FMOD nabízí pokročilé funkce jako pocit 3D prostoru, automatizace zvuku nebo nejrůznější zvukové efekty (např. reverb).

Poznatky z game jamu jsem následně aplikoval i v maturitní práci, jelikož FMOD výrazně zjednodušuje správu zvukových efektů a umožňuje mi oddělit audio design od programování.

3.5 Ostatní

Pro efektivní spolupráci v týmu používáme poznámkový software Obsidian, u kterého jsme si společně koupili předplatné Obsidian Sync, což nám zefektivňuje práci jak každodenně, tak například i na game jamech. Toto předplatné nám umožní ve stejný čas přistupovat ke všem našim informacím, nápadům, řešením, skriptům a zapisovat nové, aniž bychom si museli cokoli manuálně přeposílat přes komunikační médium.

Pro komunikaci používáme platformu Discord, kde máme založený server pod názvem Still Thinking Studio, na kterém jsou všichni členové týmu i spolupracující zvenčí. Discord nám umožňuje psát si jako v každé běžné chatovací aplikaci, spravovat desítky různých textových a hovorových kanálů (oprávnění, přístup, kategorie...) a rychle si posílat obrázky, zvukové soubory nebo videa pro nápady a zpětnou vazbu od ostatních členů týmu.

4 Celková koncepce řešení

4.1 Game Manager

Videohry se obvykle skládají z různých scén, které mohou představovat jednotlivé levely, tutorial část, hlavní menu nebo závěrečné titulky. Kdyby videohry musely načítat všechny tyto scény najednou, zabralo by to zbytečně moc času a výpočetní síly i přesto, že se hráč nachází jen na jednom z těchto míst. To je jeden z příkladů optimalizace a logiky, které jsou klíčové pro příjemný chod videohry.

Na jednom game jamu se našemu týmu stalo, že jsme měli navrhnutých více levelů, ale na game manager jsme se vrhli příliš pozdě, vyskytly se problémy a tak jsme do finálního výsledku dané levely nakonec nemohli dostat a čas strávený prací nad těmito levely byl „k zahození“. Proto jsem se do této důležité části ponořil důkladněji, abychom naším časem a zdroji nemuseli plýtvat.

Game managerů se v herních projektech vyskytuje hned několik a každý má na starosti odlišnou funkční část videohry: Game Manager (stará se o přepínání mezi scénami, správné zmrazení času, jednoduché uživatelské rozhraní...), Audio Manager (zajišťuje správný chod veškeré logiky zvuku v prostředí hry nebo v UI), Cursor Manager (upravuje vlastnosti kurzoru počítače pro účely videohry) a další managery, které nejsou zas tak časté, ale mohou se hodit pro konkrétní případy (Tutorial Manager, Collectible Manager, DontDestroyOnLoad, Death Manager).

Game Manager je (zejména v Unity) tedy „empty GameObject“ ve scéně, který má na sobě připojený stejnojmenný skript jako komponent, do kterého můžeme přidávat reference k dalším objektům ve scéně nebo v projektu (tělo hráče, další scéna, textové pole) a upravovat jakékoli parametry tohoto skriptu, které zpřístupníme Unity Editoru.

4.2 Umírání

Naši hru jsme navrhli tak, že každý level má dvě části: *The Chase* a *The Exploration*. V *The Chase* je hráč naháněn masivní bowlingovou koulí, která, pokud hráče dožene a přejede ho, tím ho i zabije a hráč bude muset začít znovu. Toto je způsob smrti *hard death*, kdy se po zabití hráče znovu načte celý level, všechny parametry se restartují a hráč si tak musí *The Chase* zapamatovat celou nazpaměť včetně všech překážek.

The Exploration využívá více milosrdnou *soft death*, kdy, pokud hráč spadne nebo se mu nějak jinak povede zemřít, místo toho, aby se scéna načetla od začátku, tak je hráč jen přemístěn k poslednímu aktivovanému checkpointu. Můžeme říct, že naše videohra tak používá hybridní systém smrti.

4.2.1 Checkpoint System

Do naší videohry je dobrý nápad zakomponovat *Checkpoint System*, který zajistí, že pokud hráč bude umírat v pozdějších fázích daného levelu, nemusí celým levellem procházet znovu, ale hra ho

transportuje k nejbližšímu checkpointu, který si uložil.

Některé hry používají i auto-save systém, kdy hra ukládá progres skoro každým momentem, tudíž kdykoli videohru můžete vypnout, zapnout a objevit se na tom samém místě, kde jste hru opustili, avšak to je ideálnější spíš pro videohry žánru open-world a naší hře bohatě stačí, pokud hráč bude docházet k checkpointům a aktivovat je.

4.3 Emotion System

Pro zpestření herního zážitku naší hry jsem přišel s konceptem *Emotion System*. Hráč bude muset v části *The Exploration* sbírat Conversation Fragments (části konverzace), které obsahují atributy emocí, a na základě kontextu celé konverzace musí hráč správně určit, jestli má daný Conversation Fragment pozitivní, negativní nebo neutrální emoci. Podle toho pak bude potřeba sesbírané Conversation Fragments správně umístit na „The Altar“ (oltář), který je schopen aktivovat portál do dalšího levelu v případě správné kombinace.

Díky tomuto konceptu naše hra obsahuje i mírný puzzle-solving a integraci příběhu, které posílí atmosféru surrealismu, kterého chceme v naší videohře docílit. Je totiž důležité, aby videohra neobsahovala pouze akci, ale i části, kde si hráč může oddechnout a připravit se na další akci. Stejně jako v hudbě, kde je složit komplikované mít majestátní, hlasité momenty, pokud správně nepracujete i s tichem. Všechno je to o kontrastu.

4.4 UI

Velkou kapitolou, která musí být v nějakém měřítku v každé videohře, je uživatelské rozhraní (dále také anglická zkratka UI). Jakákoli tlačítka, nastavení herního zážitku nebo interakce se světem videohry musí projít přes skripty, které řeší uživatelské rozhraní.

Jedna z nejzákladnějších částí UI je Input System, který řeší, jaké hardware vstupy způsobují jakou akci ve videohře. Dvě základní vstupní zařízení, kterými dnešní počítače disponují, jsou klávesnice a myš. Některé videohry používají pouze klávesnici (Hollow Knight od studia Team Cherry) a některé zas pouze myš (Happy Game od studia Amanita Design). Častěji se však můžeme setkat s kombinací obou. Ve vývoji naší videohry jsme se ještě úplně nerozhodli na definitivním způsobu ovládání naší hry, takže zatím naše hra po levé ruce vyžaduje veškerý pohyb, včetně skákání a dashe (výbušný posun kupředu), zatímco pravá ruka zaujímá pozici na myši, kterou využívá pro míření, střelení a „camera peeking“ (vysvětleno později).

Poté, co videohra ví, jaké vstupy mají jaký účel, potřebujeme s těmito signály pracovat a používat je pro kontroly, mechaniky a orientaci v programu videohry. Nejčastější případ UI jsou panely, které mají různé účely. Například Pause Panel je lišta, která se objeví při pozastavení hry (nejčastěji stisknutím klávesy Escape), kde se mohou objevovat informace o aktuálním stavu hráče, nastavení hlasitosti zvuku nebo možnosti opuštění levelu a ukončení hry.

Dokonce hned první věc, kterou hráč po zapnutí videohry vidí, je nejčastěji obrazovka hlavního menu, která se ve většině případech skládá pouze z UI komponentů. Častá jsou tlačítka „Hrát“, „Nastavení“, „Ukončit“ a „Titulky“ s pozadím, názvem herního titulu a Main Menu hudbou hrající na pozadí.

4.5 Vizuální efekty

Vizuální efekty by, jako každá část, měly podpořit uvěřitelnost digitálního světa a angažovanost do něj. Neměly by na sebe lákat až moc pozornosti, pokud tak vyloženě nechceme. Příklady klasických vizuálních efektů ve hrách jsou:

efekt smrti (Death Screen) impact effect (například dopad na zem, dash) glow effect (na hráči, aby viděl kolem sebe, nebo na jiných předmětech, aby byly rychle zaregistrovatelné)

Dosáhnout těchto efektů můžeme různými způsoby: přidání prvků světla na určitý GameObject, experimentování s Unity Particle System nebo manipulování určitých prvků Post-processing.

4.6 Audio

4.6.1 OST

OST (original soundtrack), neboli hudební složka videohry, je část vývoje her, do které jsem osobně zapálen nejvíce, jelikož práce s hudbou a zvukem je mým dlouholetým zájmem.

Skládat hudbu pro hru je samozřejmě více kreativní postup, kdy mohu strávit hodiny sezením u el. keyboardu nebo vybíráním či tvorbou ideálních zvuků, které mám v hlavě. Hudba většinou podporuje již hotovou vizuální a příběhovou stránku videoher (nebo i filmů) – málokdy se stává, že se svět videohry staví kolem hudby. Naše videohra má surreální, snovou a tajemnou atmosféru – sleduje příběh postavy, která hledá sebe sama, má neustálý strach, neví, co může čekat od nastávající chvíle, a chce se dostat do bezpečí. Během této cesty se setkává s nadpřirozenými jevy, které nedávají smysl. Je v neustálém stavu zmatení.

Pro tato témata jsem si vybral následující sonické elementy:

- vzdušné, retro tóny inspirované interpretem Joe Hisaishim (populární japonský skladatel pro legendární filmy od Studio Ghibli), zejména skladbou „The Path of the Wind – Instrumental“
- podivné reverb & delay efekty
- mixolydický modus, dórský modus, ukrajinská dórská / rumunská mollová stupnice
- mikrotonální hudba, xenharmonie
- vinylový efekt

- Další inspiraci jsem si vzal například ze skladeb „It’s Just a Burning Memory“ (The Caretaker), „Moog City 2“ (C418) a „Resonance“ (Home).

4.6.2 SFX

SFX (sound effects), neboli zvukové efekty, jsou dnes brány už jako samozřejmost a lidé si tolik neuvědomují, kolik práce na pozadí odvádí. Tvorba zvukových efektů do videohry je tedy další velká část kreativní činnosti, kde mám za úkol představit si, jak celý svět zní, a vytvořit příslušný zvukový efekt pro každou věc, která dělá nějaký zvuk. Sem spadají ambience na pozadí (padání kapek v jeskyních, šumění listů ve vánku, šeptání lidí), zvuky konkrétních objektů / akcí (kroky, útok, aktivace portálu, sbírání předmětů, dopad na zem, simulace komunikace s ostatními postavami) a zvuky uživatelského rozhraní (zmáčknutí tlačítka, pozastavení hry, hover akce).

Zvukové efekty tak můžeme rozdělit do dvou částí: diegetické a nediegetické. Diegetické jsou součástí prostředí hry, mohou je slyšet postavy ve hře, zatímco nediegetické jsou určené pro hráče, nespádají do prostředí hry a mají čistě funkční účely (zpětná vazba, indikátory...). V některých případech se v médiích používají i přechody mezi nediegetickými a diegetickými zvuky, které také mohou mít svůj význam. Můj oblíbený případ tohoto efektu je z televizního seriálu Arcane, kde v první půlminutě této video ukázky můžeme slyšet, jak se hudba vycházející z balónu (prostředí světa) postupně přemění na podkladovou hudbu pro diváky.¹

4.7 Post-processing

Post-processing dodává videohře atmosféru, šmrnc a oživuje její již zhotovenou grafickou stránku. Pracuje se světlem, stínováním, barvami, zrcadlením, šumem a dalšími efekty, které najdete ve většině grafických nástrojů. Tyto efekty jsou pak aplikovány jako poslední grafická vrstva, než obraz dojde na obrazovku, a mohou být zpřístupněny přes skripty a ovládány tak, aby reagovaly na události ve videohře. Mezi časté použití post-processingu patří:

- práce s jasnem (iluze fáze noci, realistické, intenzivní zesvětlení prostředí simulující explozi nukleární bomby...)
- saturace / desaturace obrazu pro vyzdvižení emocí
- VHS efekt pro videohry žánru „analogový horor“

¹ Arcane: Season 2 | Ekko and Jinx Team Up (Nerd Clips HD). https://youtu.be/B_KSH6w-gps

5 Popis aplikace

5.1 Zpracování základního modelu hry

5.1.1 Struktura scén

V Unity představují scény základní prostředí, ve kterých videohra operuje. Mohou to být například jednotlivé úrovně, avšak jejich využití je širší – mohou rovněž sloužit jako hlavní menu, nastavení, nebo jakékoliv jiné samostatné prostředí. V rámci této práce pracujeme se třemi základními scénami: *MainMenu*, *Level1* a *Level2*.

První scénou, se kterou se hráč setká po spuštění aplikace, je hlavní menu (*MainMenu*). To nabízí několik základních možností: vstup do herního prostředí, přizpůsobení nastavení nebo ukončení aplikace. Logiku k přechodu z menu do první úrovně řeším ve skriptu *UIManager.cs* pomocí metody *PlayGame()*, která využívá vestavěnou třídu *SceneManager*:

```
1 using UnityEngine;
2 using UnityEngine.SceneManagement;
3
4 public class UIManager : MonoBehaviour
5 {
6     private void PlayGame()
7     {
8         SceneManager.LoadScene("Level1");
9     }
10 }
```

Kód 5.1: *UIManager.PlayGame()*

Pro práci se scénami v Unity je nutné nejprve přidat požadované scény do seznamu scén v tzv. *Build Settings*. V Unity editoru se tento seznam nachází v menu *File > Build Settings > Scene List*. Scéna musí být před přidáním otevřena v editoru a následně přidána pomocí tlačítka *Add Open Scenes*. Po přidání lze na scénu odkazovat buď pomocí jejího indexu v seznamu, nebo pomocí jejího názvu. Já preferuji odkazování názvem pro lepší čitelnost kódu.

Po úspěšném dokončení úrovně *Level1* je hráč automaticky přesunut do *Level2*. Z jakékoliv úrovně se hráč může vrátit do hlavního menu pozastavením hry a aktivací panelu *Pause Menu*, což je podrobněji popsáno v podkapitole [5.2.2](#).

5.1.2 Herní manažery

Herní manažery představují klíčové komponenty architektury videohry, které řídí specifické herní systémy a zajišťují komunikaci mezi různými částmi aplikace. V aktuální verzi využívám tři hlavní

manažery:

- *UI Manager*
- *Audio Manager*
- *Cursor Manager*

UI Manager je zodpovědný za veškerou logiku související s uživatelským rozhraním. Kromě dříve zmíněného přepínání mezi scénami řeší aktivaci jednotlivých panelů (například pause menu nebo nastavení) a komunikaci s knihovnou UI Toolkit, která je podrobněji popsána v podkapitole [5.2.1](#).

Audio Manager spravuje zvukovou složku aplikace. Jeho úkolem je aktivace odpovídající hudby, zvuků vztahujících se k danému prostředí (ambience) a především komunikace s externím nástrojem FMOD Studio. Součástí *Audio Manageru* je také objekt *FMODEvents*, který propojuje konkrétní zvuky vytvořené v databázi FMOD Studio s rozhraním Unity editoru a zpřístupňuje jednotlivé zvukové soubory pro manipulaci v kódu.

Cursor Manager je funkčně nejjednodušším manažerem v celé struktuře. Jeho jedinou funkcí je vizuální kontrola kurzoru myši ve hře. V aktuální implementaci jsou používány dva stavy: normální kurzor a kurzor při stisku tlačítka. Zde je ukázka základní struktury tohoto skriptu:

```
1 using UnityEngine;
2
3 public class CursorManager : MonoBehaviour
4 {
5     [SerializeField] private Texture2D normalCursorTexture;
6     [SerializeField] private Texture2D clickedCursorTexture;
7
8     private bool isClicking = false;
9
10    private void Update()
11    {
12        bool currentlyClicking = Input.GetMouseButton(0);
13
14        if (currentlyClicking != isClicking)
15        {
16            isClicking = currentlyClicking;
17            UpdateCursorSprite();
18        }
19    }
20
21    private void UpdateCursorSprite()
22    {
```

```

23         Texture2D currentTexture = isClicking ? clickedCursorTexture :
           normalCursorTexture;
24
25         if (currentTexture == null)
26         {
27             Cursor.SetCursor(null, Vector2.zero, CursorMode.Auto);
28             return;
29         }
30
31         Cursor.SetCursor(currentTexture, Vector2.zero, CursorMode.Auto)
           ;
32     }
33 }

```

Kód 5.2: CursorManager.cs

Skript využívá metodu `DontDestroyOnLoad`, aby instance manažeru přetrvávala mezi scénami. V metodě `Update()` se každým snímkem kontroluje stav levého tlačítka myši – při změně stavu se zavolá `UpdateCursorSprite()`, která nastaví odpovídající texturu kurzoru.

Na Linuxu se mi vyskytly problémy se zpožděním kurzoru, proto skript obsahuje dodatečná složitější řešení z internetu, která na něm zajišťují lepší odezvu.

5.1.3 Input systém

Input systém je jedním z více témat, ke kterému se dá přistupovat více způsoby. Můžeme ho řešit nízkoúrovňově, jako v tomto případě, kdy nasloucháme stisknutí kláves přímo pomocí C#:

```

1 if (Event.current.isKey)
2 {
3     // klávesa, která byla stisknuta.
4     KeyCode key = Event.current.keyCode;
5 }

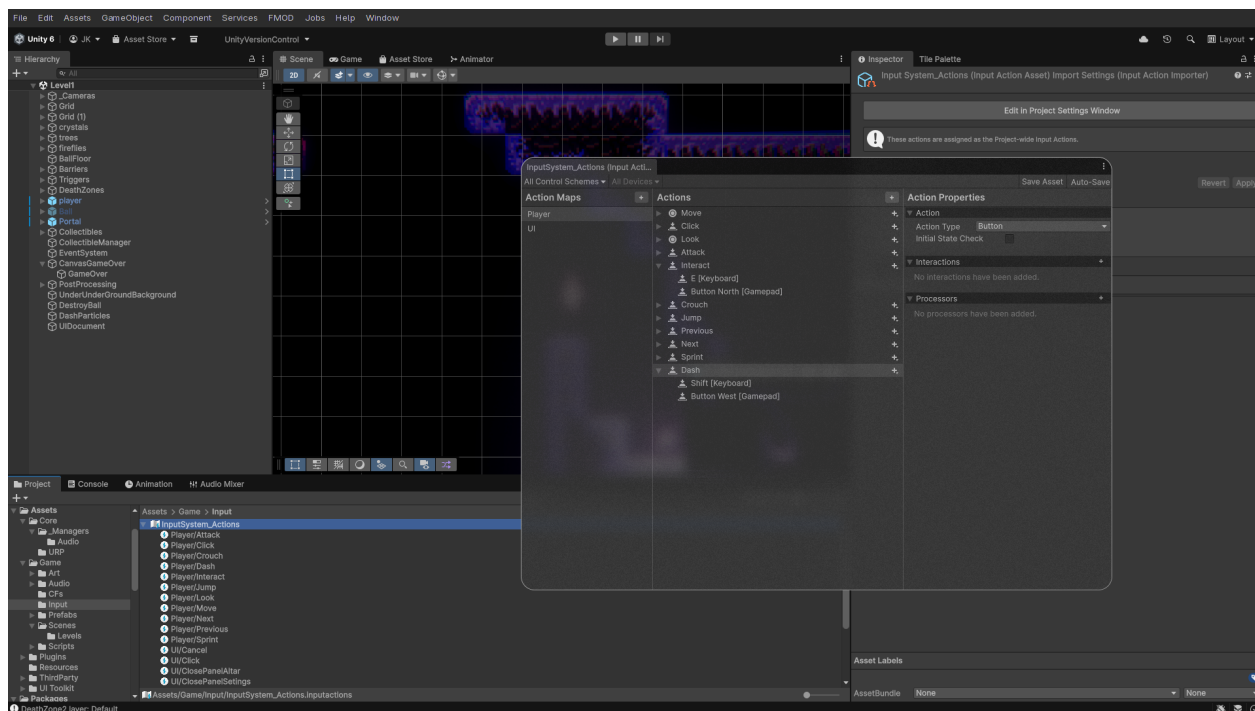
```

Kód 5.3: Nízkoúrovňové zpracování vstupu

Pro tento účel však existuje modernější systém, který zajišťuje mnohem přívětivější a flexibilnější workflow. Je jím *Input System*, který je zobrazen na obrázku 5.1.

Jak můžeme vidět na obrázku 5.1, v tomto rozhraní můžeme vytvářet *Action Maps* (vlevo). Naše videohra momentálně využívá dvě *Action Maps*, jednu pro herní mechaniky a druhou pro orientaci v uživatelském rozhraní. Toto velmi usnadňuje řešení toho, jaké akce chceme od hráče vyžadovat.

Například jedním z nepříjemných problémů, které *Action Maps* řeší, je následující: V minulosti, když jsem vstup řešil ještě zastaralým způsobem, UI fungovalo tak, že i když hráč pozastavil hru a dostal se do *Pause Menu*, čas se zastavil – viz ukázka kódu 5.4.



Obrázek 5.1: Rozhraní Unity Input System

```

1 private void SetPauseState(bool pause)
2 {
3     if (pauseMenu != null)
4         pauseMenu.SetActive(pause);
5
6     if (pause)
7     {
8         Time.timeScale = 0f;
9         AudioManager.instance.PauseMusic();
10    }
11    else
12    {
13        Time.timeScale = 1f;
14        AudioManager.instance.ResumeMusic();
15    }
16 }

```

Kód 5.4: Původní řešení pozastavení hry

Toto řešení zastavilo veškerý pohyb ve hře. Všechny vstupní akce však zůstaly povolené a byly rozházené po projektu ve skriptech pro ovládání hráče, střelbu nebo animace. Proto, i přes zastavený čas, když hráč například spustil akci pro střelbu, i v *Pause Menu* se hráči objevila střela u těla a hned jak hráč hru znovu spustil, střela odpalovala. Nebo pokud se hráč pokoušel hýbat v *Pause Menu*,

fyzicky se hýbat nemohl, ale animace fungovaly a hráč se tak mohl stále otáčet ze strany na stranu. To vypadalo neprofesionálně.

Za pomoci *Action Maps* však složité ošetřování těchto chyb rychle mizí – viz ukázka 5.5.

```
1 private void SetPauseState(bool pause)
2 {
3     if (pause)
4     {
5         Time.timeScale = 0f;
6         playerActionMap.actionMap.Disable();
7         uiActionMap.actionMap.Enable();
8     }
9     else
10    {
11        Time.timeScale = 1f;
12        uiActionMap.actionMap.Disable();
13        playerActionMap.actionMap.Enable();
14    }
15 }
```

Kód 5.5: Moderní řešení pozastavení pomocí Input System

Přes skript `UIManager.cs` mohu k *Input Actions* souboru přistoupit a deaktivovat celou jednu *Action Map*, která obsahuje jednotlivé *Input Actions*. To znamená, že pokud hráč pozastaví hru a dostane se do *Pause Menu*, mohu jednoduše zakázat *Action Map* s názvem *Player*, která obsahuje všechny možnosti ovládání herních mechanik, jako pohyb, speciální schopnosti nebo interakce se světem, a aktivovat *Action Map* s názvem *UI*, protože hráč se vskutku v daném momentu nachází v panelu uživatelského rozhraní, kde potřebuje pouze tyto *Input Actions* včetně stisknutí klávesy *Esc*, která deaktivuje *Pause Panel*, vrátí hráče do hry, deaktivuje *Action Map* pro *UI* a aktivuje *Action Map* pro ovládání hráče.

Napravo od sekce *Action Maps* se nachází sekce *Actions*, kde již nastavujeme konkrétní *Input Actions*. Ve verzi pro tuto maturitní práci využívám pro *Action Map* hráče následující akce:

- **Move** – pohyb (WASD / šipky)
- **Attack** – střelba (K / Enter)
- **Interact** – interakce s objekty (E)
- **Jump** – skok (Space)
- **Dash** – rychlý úskok (Shift)
- **Click** – kliknutí myší pro střelbu

Pro *Action Map* uživatelského rozhraní pak:

- **Click / Point** – kliknutí a pozice kurzoru myši
- **ScrollWheel** – scrollování
- **TogglePause** – pozastavení hry (Esc)
- **ClosePanelSettings / ClosePanelAltar** – zavření nastavení nebo oltáře (Esc / E)

Ke každé *Input Action* lze v sekci *Action Properties* nastavit typ akce (*Button*, *PassThrough*, *Value*), cílovou platformu (klávesnice, myš, herní ovladač, VR ovladač...) a případně další vlastnosti. Toto umožňuje multiplatformní nasazení na PC, macOS, PlayStation 5, Xbox Series X/S nebo VR headsety bez nutnosti měnit kód.

”

5.1.4 Základní herní objekty

Mezi hlavní prvky ve scéně patří:

Kamera – díky ní může hráč vidět, co se děje. S kamerou je možno dělat mnoho věcí, na které se podíváme později.

GameObject hráče – objekt, na kterém je renderován sprite hráče a animace (viz obrázek 5.3). Jsou na něm připnuty skripty jako `PlayerController.cs`, sloužící pro základní kontroly jako je pohyb, zvukový emitér atd. V naší hře je objekt hráče vždy na očích v oblasti středu okna aplikace.

Uživatelské rozhraní – to může být řešeno více způsoby. Způsob, který najdeme ve většině tutoriálů, je *Canvas* (viz podkapitola 5.2.1). V hierarchii Unity se přidá *GameObject Canvas* a do něj lze přidávat jednotlivé UI elementy jako další objekty (tlačítka, slidery, panely, text).

Na již dříve zmíněném Brackeys Game Jam 2026.1 (viz podkapitola 5.4.1) jsem kromě jiného přišel k lepší alternativě i zde – *UI Toolkit*. Toto řešení vyžaduje mít ve scéně *UI Document* (kde se řeší struktura) a *UI Manager*, který s ním komunikuje a binduje k němu ostatní skripty.

Hierarchie scény je zobrazena na obrázku 5.2 úplně vlevo.

5.2 Vytvoření uživatelského rozhraní

5.2.1 UI Toolkit

UI Toolkit je relativně novější způsob řešení uživatelského rozhraní než původní systém *Canvas*. Od něj se liší například tím, že pro sestavení a stylizaci vizuálních elementů nepoužívá klasickou *GameObject* strukturu, ale jazyky UXML (struktura) a USS (stylizace), které se svým provedením více podobají webovému vývoji.

Psát UXML nebo USS kód však vůbec není nutné. UI Toolkit má své grafické rozhraní, ve kterém je možné elementy přidávat myši a v inspektoru jim upravovat různé parametry jako pozici,

musí být také.

UI Toolkit však nepoužívá singleton logiku na *UIDocument* – ten není omezen na jednu scénu a je globální, podobně jako prefaby. Místo toho je singleton vzor aplikován na *UIManager*, který drží reference na jednotlivé panely a spravuje jejich stav pomocí *GameState* enumu. Tím se UI odděluje od datové vrstvy – struktura UI je definována v UXML, zatímco logika je soustředěna v jednom centrálním skriptu.

Strukturu uživatelského rozhraní lze rozdělit na dvě hlavní kategorie: UI a HUD. UI je obecný pojem pro všechny elementy uživatelského rozhraní, se kterými hráč může interagovat – tlačítka, inventář, textové oblasti dialogů. Naproti tomu HUD (Heads-Up Display) se skládá z elementů, které jsou perzistentně zobrazovány na obrazovce hráče během hraní – ukazatel zdraví, počítadlo nábojů nebo malá mapa na kraji obrazovky.

Jak jsem postupně přecházel z *Canvas* na UI Toolkit, mohl jsem porovnat obě řešení. Ve starších verzích bylo UI postaveno na principech:

- Více samostatných *Canvas* objektů ve scéně (např. *CanvasGameOver*, *Canvas HUD*, *CanvasTutorial*)
- Každý panel jako samostatný *GameObject* s komponentami jako *Image*, *Button*, *Text*
- Skrývání a zobrazování panelů pomocí *GameObject.SetActive(true/false)*
- Rozptýlená logika mezi skripty jako *StripeCounterUI.cs*, *OrbCounterUI.cs*, *ButtonEffects.cs*

S UI Toolkit v nejnovější verzi přichází zjednodušení:

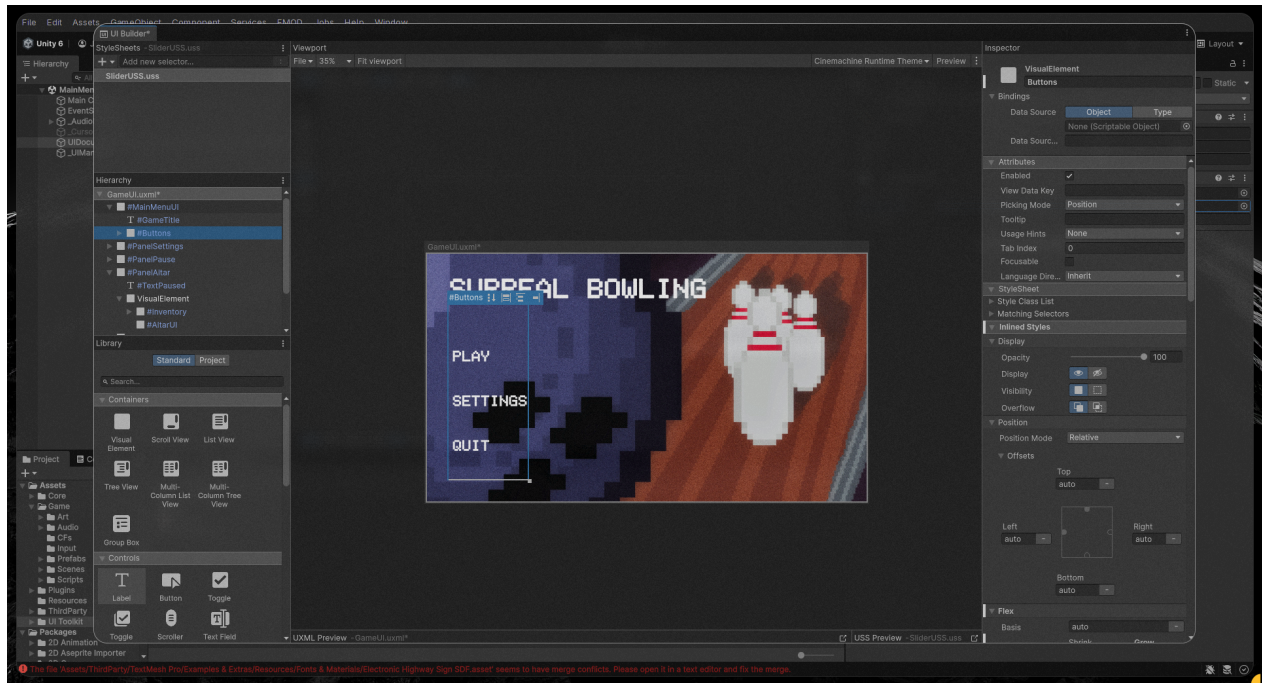
- Jeden *UIDocument* definuje veškeré UI v jednom UXML souboru
- Všechny panely (*MainMenu*, *PanelSettings*, *PanelPause*, *PanelAltar*, *HUD*) jsou dotazovány pomocí `root.Q<VisualElement>("NazevPanelu")`
- Skrývání a zobrazování pomocí `VisualElement.style.display = DisplayStyle.None/Flex`
- Centralizovaná správa stavu pomocí *GameState* enumu (*None*, *Gameplay*, *Altar*, *Pause*)

Rozhraní UI Toolkit je zobrazeno na obrázku 5.4.

5.2.2 UI Prvky

Na centrálním *UIManageru* je postavena veškerá logika jednotlivých UI prvků. Každý prvek je definován v UXML souboru a následně bindován k odpovídajícím skriptům.

Pause Menu – obsahuje tlačítko pro návrat do hlavního menu. Jeho řízení je součástí *GameState* systému, nikoliv pouze přes *Time.timeScale*. Metoda *SetState()* přepíná mezi stavy a při vstupu do *Pause* zároveň aktivuje *UI Action Map* a deaktivuje *Player Action Map*, což je podrobně popsáno v podkapitole 5.1.3. Přejít mezi stavy *Gameplay*, *Pause* a *Altar* řeší jediná metoda:



Obrázek 5.4: Rozhraní UI Toolkit

```

1 void SetState(GameState newState)
2 {
3     if (CurrentState == newState) return;
4     CurrentState = newState;
5
6     panelPause.style.display = DisplayStyle.None;
7     panelAltar.style.display = DisplayStyle.None;
8     HUD.style.display = DisplayStyle.None;
9
10    switch (newState)
11    {
12        case GameState.Gameplay:
13            Time.timeScale = 1f;
14            ActivatePlayerInput();
15            HUD.style.display = DisplayStyle.Flex;
16            break;
17        case GameState.Altar:
18            panelAltar.style.display = DisplayStyle.Flex;
19            ActivateUIInput();
20            Time.timeScale = 1f;
21            HUD.style.display = DisplayStyle.None;
22            break;

```



```

23         case GameState.Pause:
24             panelPause.style.display = DisplayStyle.Flex;
25             ActivateUIInput();
26             Time.timeScale = 0f;
27             HUD.style.display = DisplayStyle.None;
28             break;
29     }
30 }

```

Kód 5.6: UIManager – přepínání stavů pomocí GameState enumu

Settings Panel – obsahuje slidery pro ovládání hlasitosti jednotlivých zvukových kanálů. Tyto slidery jsou bindovány na FMOD mixer prostřednictvím AudioManageru, který uchovává reference na FMOD *Busy* pro master, hudbu, ambience a SFX. Propojení UI Toolkit sliderů s FMOD mixerem:

```

1 private void BindVolumeSliders(VisualElement root)
2 {
3     masterSlider = root.Q<Slider>("SliderVolumeMaster");
4     masterSlider.SetValueWithoutNotify(AudioManager.instance.
        masterVolume);
5     masterSlider.RegisterValueChangedCallback(evt =>
6     {
7         AudioManager.instance.masterVolume = evt.newValue;
8         AudioManager.instance.SaveVolumeSettings();
9     });
10 }

```

Kód 5.7: Bindování sliderů k FMOD mixeru

Hodnoty jsou ukládány do PlayerPrefs, takže při dalším spuštění hry jsou obnoveny.

Nastavení hlasitosti je zobrazeno na obrázku 5.5.

HUD – zobrazuje informace perzistentní během hraní. V levém horním rohu je počítadlo zbývajících střel (stripe shot), v pravém horním rohu pak počet nasbíraných konverzačních fragmentů (CFs). Binding počítadel probíhá přímým dotazem na herní skripty – například `CFInventory.OnCFCollected` událost aktualizuje label v HUD:

```

1 private void BindCFCounter(VisualElement root)
2 {
3     cfCounterLabel = root.Q<Label>("LabelCFCount");
4     cfInventory = FindFirstObjectByType<CFInventory>();
5
6     if (cfInventory != null)
7         cfInventory.OnCFCollected += OnCFCollected;
8 }
9

```

```

10 private void OnCFCollected(int count)
11 {
12     cfCounterLabel.text = $"CFs: {count}";
13 }

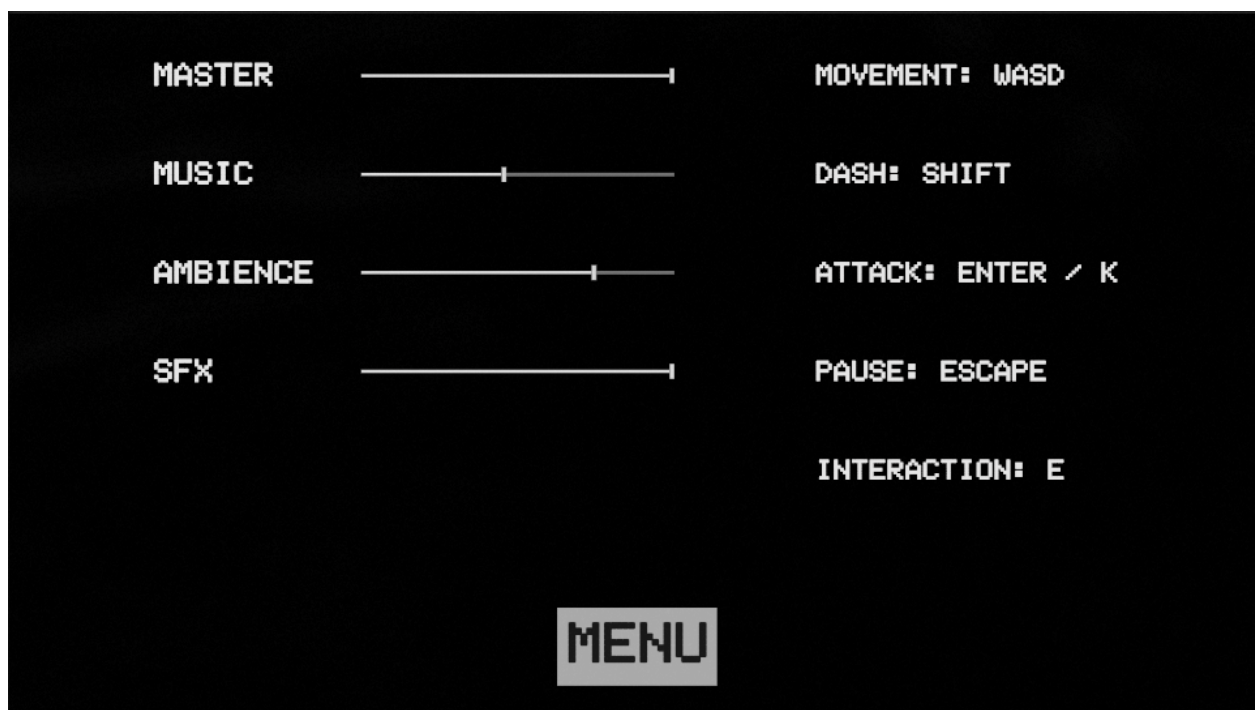
```

Kód 5.8: Bindování CF počítadla

Podobně funguje i `StripeCounterUI`, který naslouchá událostem z herních skriptů a aktualizuje text v UI. Rozhraní HUD je zobrazeno na obrázku 5.6 – levý horní roh znázorňuje počet střel a pravý horní roh počet sesbíraných CFs.

Altar / Inventory UI – panel oltáře slouží jako inventář nasbíraných předmětů a konverzačních fragmentů. Zatím je ve fázi vývoje a nelze jej považovat za hotový. Struktura se skládá z mřížky inventáře a obrázku oltáře. Otevření panelu je vyvoláno interakcí hráče s objektem oltáře ve scéně.

Jakmile bude tato sekce plně implementována, umožní hráči procházet nasbíranými předměty a interagovat s nimi.

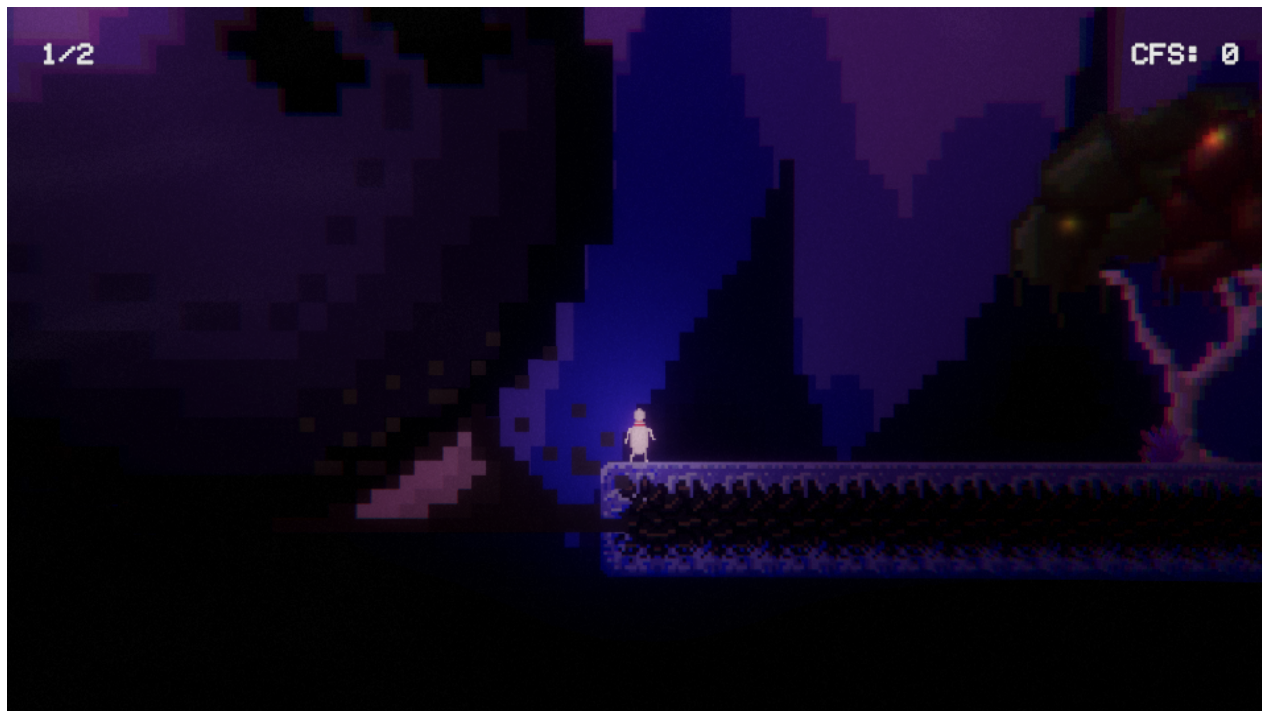


Obrázek 5.5: Nastavení hlasitosti

5.2.3 Ostatní funkce UI

Kromě hlavních panelů a HUD prvků popsanych v předchozí podkapitole existuje v projektu několik dalších vizuálních komponent, které doplňují herní zážitek.

Respawn / Checkpoint systém – systém checkpointů umožňuje hráči ukládat pozici, na kterou bude respawnován po *soft death* (používáno v *The Exploration*). Checkpoint je objekt ve scéně,



Obrázek 5.6: HUD ve hře – levý horní roh: počet střel, pravý horní roh: počet CFs

s nímž hráč může interagovat stisknutím klávesy E. Při aktivaci se uloží jako referenční bod v komponentě *DeathZoneSoft*, která po vstupu hráče do svého triggeru okamžitě přesune hráče na pozici posledního aktivovaného CheckPointu:

```

1 public void Activate()
2 {
3     if (lastActivated != null && lastActivated != this)
4         lastActivated.Deactivate();
5     lastActivated = this;
6     deathZoneSoft.respawnPoint = this.gameObject;
7     spriteRenderer.color = Color.green;
8 }

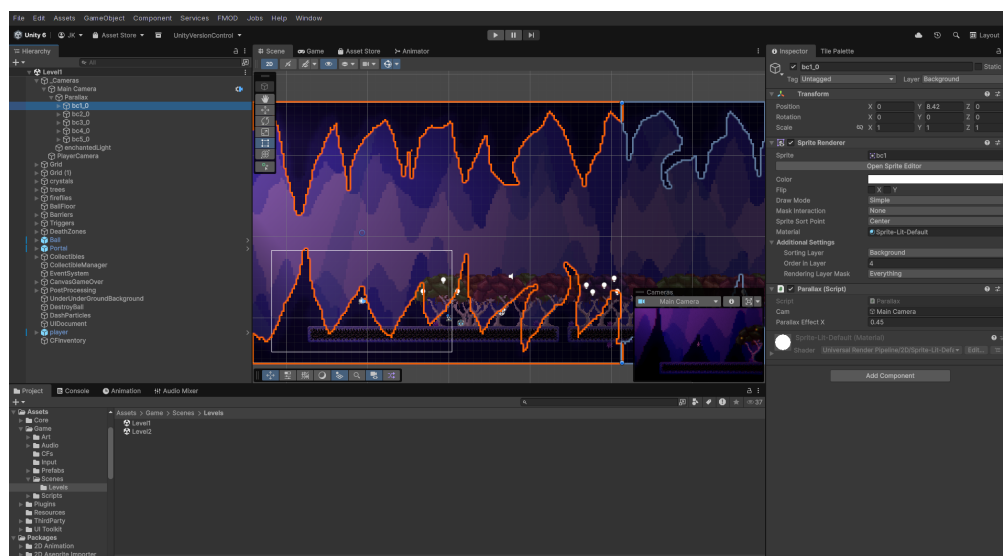
```

Kód 5.9: CheckPoint – aktivace checkpointu

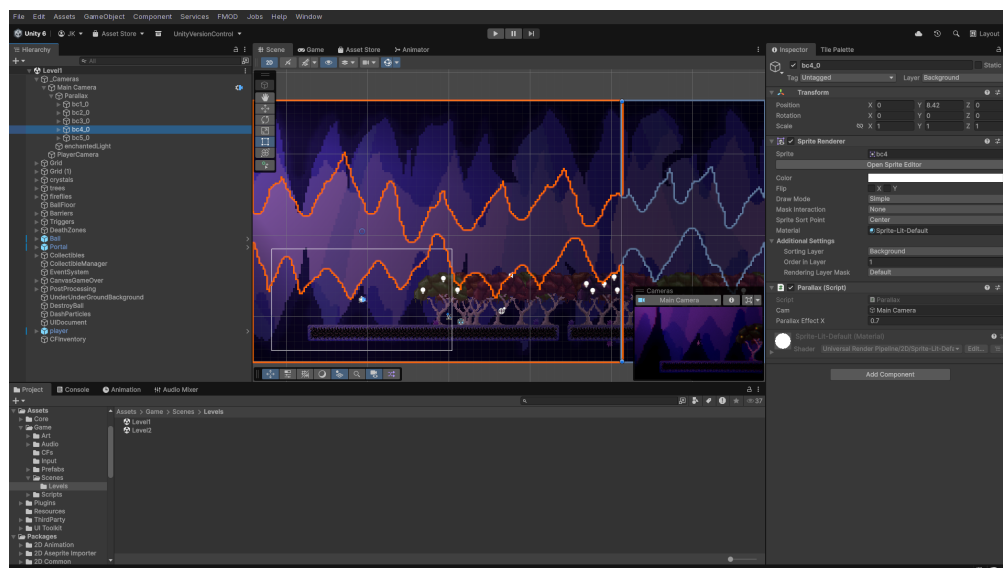
V aktuální implementaci se při *soft death* přesouvá pouze pozice hráče – počet střel i *conversation fragments* zůstávají zachovány. Toto odpovídá designovému rozhodnutí pro *The Exploration*, kde je kladen důraz na průzkum bez trestu. Naproti tomu *The Chase* bude implementovat *hard death*, který skrze reload celé scény obnoví všechny proměnné do výchozího stavu.

Camera Respawn Catch – komponenta zajišťující plynulé přesunutí kamery k hráči po respawnu. Při volání metody *OnPlayerRespawn()* se cílová pozice okamžitě nastaví na pozici hráče, takže kamera může pomocí interpolace (*Vector3.Lerp*) hladce dohnat případný rozestup vzniklý během teleportace.

Parallax – vizuální komponenta pozadí, která simuluje hloubku prostoru pomocí odlišných rychlostí pohybu jednotlivých vrstev. Sprite renderer na každé vrstvě dostane koeficient `parallaxEffectX`; nižší hodnota znamená, že vrstva se pohybuje pomaleji než kamera a působí vzdáleněji. Na obrázcích 5.7 a 5.8 je vidět nastavení bližší vrstvy na 0.45x a vzdálenější vrstvy na 0.7x – bližší vrstva se pohybuje výrazně pomaleji a působí staticky, zatímco vzdálenější vrstva rychleji následuje kameru, což navozuje dojem prostorové hloubky.



Obrázek 5.7: Bližší parallax vrstva (`parallaxEffectX = 0.45`)



Obrázek 5.8: Vzdálenější parallax vrstva (`parallaxEffectX = 0.7`)

Tlačítková hlasová odezva – rozšíření `UIToolkitAudioBinding` přidává k libovolnému `VisualElement` metodu `BindHoverSound()`, která registruje callback na událost `PointerEnterEvent`. Tím se

při najetí kurzoru na libovolný element (nejen tlačítka) přehraje zvuk hover efektu, což výrazně zpříjemňuje navigaci v menu a panelech:

```
1 public static class UIToolkitAudioBinding
2 {
3     public static void BindHoverSound(this VisualElement ve, System.
        Action play)
4     {
5         ve.RegisterCallback<PointerEnterEvent>(_ => play());
6     }
7 }
```

Kód 5.10: UIToolkitAudioBinding – hover zvuk

5.3 Tvorba vizuálních efektů

5.3.1 Light 2D

Pro osvětlení 2D scény využívám systém *Light 2D*, který je součástí *Universal Render Pipeline*. Na rozdíl od tradičního 3D osvětlení, kde jsou světelné paprsky vysílány ze zdrojů a odrazy osvětlují okolní prostředí, Light 2D funguje invertovaně – každý objekt ve scéně nese informaci o tom, jak má být osvětlen, a globální světlo tyto vlastnosti zprůměruje a aplikuje. Tento přístup je výrazně méně náročný na výkon a ideální pro stylizované 2D hry.

Jedním z nejdůležitějších prvků osvětlení je Light2D typu *Spot* připojený jako child object k objektu hráče. Toto řešení bylo inspirováno hrou *Ori and the Blind Forest*, kde je postava neustále obklopena jemným světelným polem, které jí umožňuje vidět v temném prostředí. Díky tomu, že je GameObject světla child objectem hráče, automaticky se pohybuje spolu s ním bez nutnosti dalšího kódu.

Kromě individuálních zdrojů světla je ve scéně přítomno také *Global Light 2D*, které nastavuje základní intenzitu a barvu osvětlení pro celou scénu. Tím se zajistí, že objekty bez vlastního zdroje světla nejsou zcela ponořeny do tmy, ale zároveň jsou dostatečně kontrastní, aby vynikla atmosféra temnějších částí levelu.

Dalším typem zdroje světla jsou *světlušky (Firefly Lights)* v korunách stromů. Každá instance komponenty FireflyLight řídí intenzitu připojeného Light2D pomocí funkce `Mathf.PingPong`, která vrací hodnotu v rozsahu 0 až 1 tak, že postupně roste a pak zase klesá, dokola – výsledkem je hladké rozjasňování a zhasínání světla. Každá instance má navíc náhodný `randomOffset` (viz ukázka kódu), takže jednotlivé světlušky pulzují nezávisle na sobě a vytvářejí dynamičtější a přirozenější efekt:

```
1 void Update()
2 {
3     float t = Mathf.PingPong(Time.time * speed + randomOffset, 1f);
```

```

4     light2D.intensity = Mathf.Lerp(minIntensity, maxIntensity, t);
5 }

```

Kód 5.11: FireflyLight – pulzující světlo

Collectible objekty (*conversation fragments*) rovněž nesou vlastní Light2D zdroj, který je činí vizuálně nápadnými a usnadňuje hráči jejich vyhledávání v prostředí.

Posledním významným případem využití Light 2D je PortalActivator, který při aktivaci portalu postupně zvyšuje intenzitu jeho světla pomocí Mathf.Lerp v čase – světlo se rozsvítí v průběhu dvou sekund spolu se sprite průhledností a hlasitostí zvuku:

```

1 private IEnumerator FadeInPortalLight()
2 {
3     Light2D portalLight = portalVisual.GetComponentInChildren<Light2D>
4         >();
5     portalLight.intensity = 0f;
6
7     float elapsed = 0f;
8     while (elapsed < fadeDuration)
9     {
10         elapsed += Time.deltaTime;
11         float progress = elapsed / fadeDuration;
12         portalLight.intensity = Mathf.Lerp(0f, targetLightIntensity,
13             progress);
14         yield return null;
15     }
16     portalLight.intensity = targetLightIntensity;
17 }

```

Kód 5.12: PortalActivator – fade in portalu

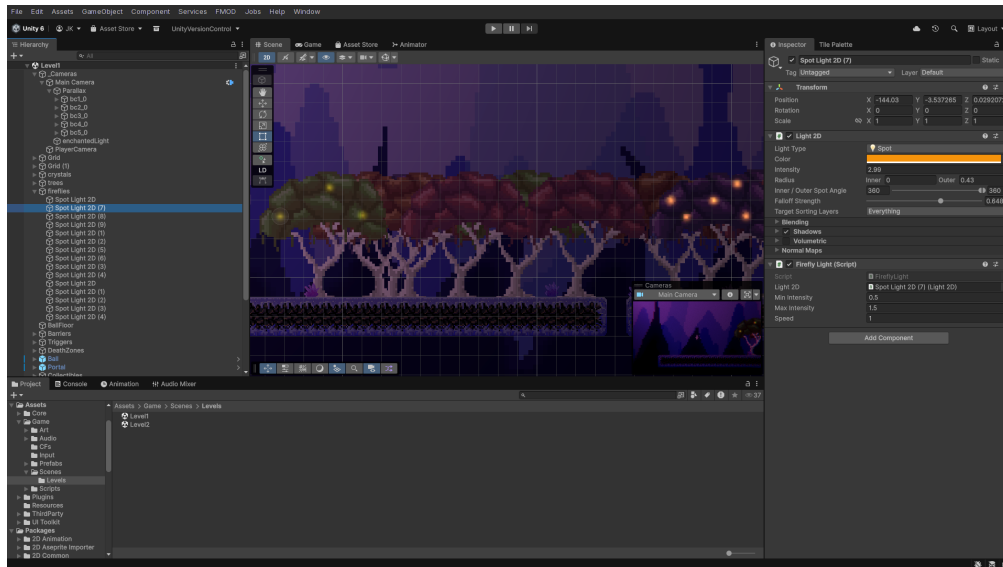
5.3.2 Particle System

Systém částic (Particle System) v Unity umožňuje vytvářet efekt velkého množství malých objektů, které se chovají jako tekutina, kouř, jiskry, nebo v tomto případě – efekt rychlého úskoku. Při provedení *dash* schopnosti je volána statická metoda PlayDashEffect(), která spustí ParticleSystem na aktuální pozici hráče. Metoda Play() automaticky restartuje animaci částic od začátku, takže není třeba částice před novým spuštěním resetovat. Třída DashParticles využívá singleton vzor, takže může být volána odkudkoliv bez nutnosti držet přímou referenci:

```

1 public static void PlayDashEffect(Vector3 position)
2 {
3     if (instance != null)
4         instance.TriggerDash(position);
5 }

```



Obrázek 5.9: Světlušky v korunách stromů ve scéně

```

5 }
6
7 private void TriggerDash(Vector3 position)
8 {
9     transform.position = position;
10    dashParticles.Play();
11 }

```

Kód 5.13: DashParticles – spuštění částic

Samotný vizuální vzhled částic je definován přímo v Unity editoru v komponentě `ParticleSystem` – barva, tvar emitru, doba životnosti jednotlivých částic a trajektorie jsou nastaveny jako asset a nejsou řízeny kódem.

5.3.3 Ostatní efekty

Dalším vizuálním prvkem je komponenta `OrbBehavior`, která ovlivňuje chování `collectible` objektů (CFs). Když se hráč nachází v blízkosti `collectible`, aktivuje se magnetický efekt – objekt se pomocí `Vector3.MoveTowards` přibližuje k hráči, dokud ho není možné sebrat. Pokud se hráč od `collectible` vzdálí, objekt přejde do režimu idle, kdy jemně klesá a stoupá na vertikální ose pomocí funkce `Mathf.Sin`:

```

1 float dist = Vector2.Distance(transform.position, player.position);
2 magnetActive = dist <= magnetRadius;
3
4 if (magnetActive)
5 {

```

```

6      transform.position = Vector3.MoveTowards(
7          transform.position,
8          player.position,
9          pullSpeed * Time.deltaTime
10     );
11 }
12 else
13 {
14     float newY = startY + Mathf.Sin(Time.time * frequency) * amplitude;
15     transform.position = new Vector3(transform.position.x, newY,
16                                     transform.position.z);
17 }

```

Kód 5.14: OrbBehavior – magnet a idle animace

Další vizuální efekt, který zasahuje do herní mechaniky, je komponenta *DeathEffect*. Ta ovlivňuje proměnné Volume profilu v rámci *Universal Render Pipeline* – konkrétně snižuje saturaci obrazu a aplikuje tmavý overlay, čímž vytváří cinematický efekt úmrtí. Jedná se o vizuální efekt, který je však implementován pomocí post-processing nástrojů, a proto je podrobněji rozebrán v kapitole [5.5](#).

5.4 Tvorba zvukových efektů

5.4.1 Audio systém

Práce se zvukem ve hrách je komplexní téma, které zahrnuje správu mnoha souběžných zvukových stop, jejich prostorové umístění, míchání hlasitostí jednotlivých kanálů a plynulé přechody mezi stavy. V rámci vývoje *Surreal Bowling: Salvation Path* jsem prošel evolucí od čistě Unity *AudioMixer* řešení k profesionálnímu nástroji *FMOD Studio*, což přineslo výrazné zlepšení v organizaci, flexibilitě i kvalitě výsledného audia.

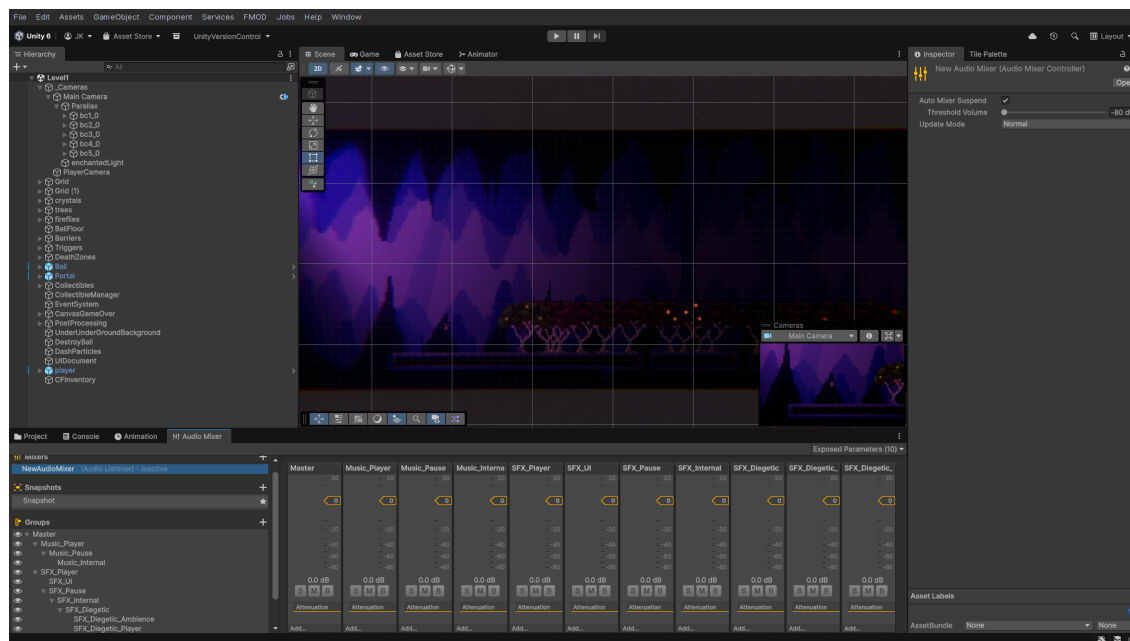
Původní řešení – Unity *AudioMixer*

V původní verzi projektu byl audio systém postaven na Unity vestavěném *AudioMixeru* a centralizovaném skriptu *AudioManager.cs*, který využíval singleton vzor. Tento manažer spravoval tři hlavní zvukové kanály – *Music*, *Diegetic* a *Ambience* – a byl perzistentní mezi scénami díky metodě *DontDestroyOnLoad*.

Hierarchie *AudioMixeru*

Hierarchie *AudioMixeru* začíná na kořenovém kanálu *Master*, pod nímž se nacházejí dva hlavní kanály: *Music_Player* a *SFX_Player* (viz obrázek [5.10](#)). Kanál *Music_Player* obsahuje podřízený kanál *Music_Pause*, který řídí hlasitost hudby při pozastavení hry, a pod ním *Music_Internal*, kam směřuje samotný *AudioSource* přehrávající hudbu. Kanál *SFX_Player* se pak větví na *SFX_UI*

pro uživatelské rozhraní, *SFX_Pause* pro diegetické zvuky při pauze, *SFX_Internal* jako centrální kanál pro všechny SFX, a ten se dále dělí na *SFX_Diegetic_Ambience* (environmentální ambientní smyčky) a *SFX_Diegetic_Player* (zvuky vztahující se k hráči).



Obrázek 5.10: Struktura Unity AudioMixeru – hierarchie kanálů

Plynulé přechody pomocí Lerp

Klíčovou funkcí původního systému byla metoda `FadeAudio()`, která zajišťovala plynulé přechody hlasitosti pomocí lineární interpolace (`Mathf.Lerp`) mezi aktuální a cílovou hodnotou v decibelech:

```
1 private IEnumerator FadeAudio(string mixerParam, float targetDB, float
    fadeDuration)
2 {
3     mixer.GetFloat(mixerParam, out float startDB);
4
5     float elapsed = 0f;
6     while (elapsed < fadeDuration)
7     {
8         elapsed += Time.unscaledDeltaTime;
9         float t = elapsed / fadeDuration;
10        mixer.SetFloat(mixerParam, Mathf.Lerp(startDB, targetDB, t));
11        yield return null;
12    }
13    mixer.SetFloat(mixerParam, targetDB);
```

14 }

Kód 5.15: AudioManager – plynulý přechod hlasitosti pomocí Lerp

Tato korutina byla využívána pro všechny operace vyžadující plynulý přechod – fade-in a fade-out hudby při přechodu mezi scénami, pozastavení hudby při `Time.timeScale = 0f`, nebo ztišení diegetických zvuků při úmrtí hráče (`MuteDiegetic()`). Hodnota `Time.unscaledDeltaTime` zajišťuje, že fade proběhne v reálném čase i při zastaveném herním čase, což je kritické pro správné fungování při pauze.

Metoda `PlaySound()` přijímala `SoundType` enum, pole `AudioClip`ů a přehrávala náhodný klip z pole. Tím se docílilo variace zvuků – například pro footsteps existovaly tři různé zvukové soubory, z nichž byl pokaždé náhodně vybrán jeden:

```
1 public static void PlaySound(SoundType sound, AudioSource audioSource,
   float volume = 1f)
2 {
3     if (instance == null || audioSource == null) return;
4
5     int soundIndex = (int)sound;
6     var clips = instance.soundList[soundIndex].Sounds;
7     var clip = clips[UnityEngine.Random.Range(0, clips.Length)];
8     audioSource.PlayOneShot(clip, volume);
9 }
```

Kód 5.16: AudioManager – přehrání zvuku s náhodnou variací

Nevýhody původního přístupu

Nevýhodou tohoto přístupu byla centralizace veškeré logiky do jednoho skriptu a absence nativní podpory pro pokročilé zvukové techniky jako je crossfading, shuffle (náhodný výběr z více variant s garantovanou unikátností) nebo prostorové zvuky. Proto jsem při přechodu na nejnovější verzi zvolil profesionální nástroj FMOD Studio.

FMOD Studio

FMOD Studio je profesionální zvukový middleware, který umožňuje návrh zvukového obsahu v dedikovaném editoru a následnou integraci do Unity. Pro každou akci ve hře lze vytvořit *Event* – kontejner, který může obsahovat jeden nebo více zvukových klipů, efekty (EQ, komprese, reverb), mixovací nastavení i parametrické ovladače. FMOD projekt (`FMOD_SB.fsp`) obsahuje tři hlavní *Banky*: *Master*, *Music* a *SFX*. Master bank obsahuje globální nastavení a mixovací cesty (busy), Music bank obsahuje všechny hudební stopy, SFX bank pak veškeré jednotlivé zvukové efekty.

Mixovací struktura v FMOD je podobná struktuře původního Unity AudioMixeru – existují čtyři hlavní busy: *Master*, *Music*, *Ambience* a *SFX*. Hudba a ambience jsou kontinuální smyčky řízené skriptem, zatímco SFX bus obsahuje všechny jednorázové zvukové efekty.

Architektura FMOD pluginu pro Unity

Integrace FMOD do Unity je zajištěna oficiálním FMOD for Unity pluginem, který přidává do Unity editoru několik komponent a editorových oken. Základní architektura FMOD v Unity se skládá z několika vrstev:

- *RuntimeManager* – inicializuje FMOD runtime při startu aplikace
- *StudioEventEmitter* – komponenta připojená k GameObjectu, která přehrává FMOD eventy na základě herních událostí
- *EventReference* – reference na konkrétní FMOD event, která se binduje v Inspectoru
- *EventHandler* – abstraktní základní třída, která mapuje Unity události (Start, OnDestroy, OnEnable, kolize, trigger) na FMOD akce

Komponenta StudioEventEmitter a EmitterGameEvent

Komponenta StudioEventEmitter je jádrem FMOD integrace v Unity. Její klíčové vlastnosti jsou:

- EventReference – odkaz na FMOD event vytvořený v FMOD Studio
- EventPlayTrigger / EventStopTrigger – určují, jaká Unity událost spustí/zastaví přehrávání
- AllowFadeout – zda má event při zastavení použít fade-out nebo se zastavit okamžitě
- TriggerOnce – zda se event může přehrát pouze jednou
- Params – pole FMOD parametrů, které lze nastavit

Enum EmitterGameEvent definuje všechny podporované Unity události, na které lze event navázat:

- ObjectStart / ObjectDestroy / ObjectEnable / ObjectDisable
- TriggerEnter / TriggerExit a jejich 2D varianty
- CollisionEnter / CollisionExit a jejich 2D varianty
- UIMouseEnter / UIMouseExit / UIMouseDown / UIMouseUp

Základní třída EventHandler (ze které StudioEventEmitter dědí) registruje callbacky na tyto Unity události a při jejich vyvolání volá HandleGameEvent(), která porovnává typ události s nastaveným EventPlayTrigger a EventStopTrigger. Při shodě pak zavolá Play() nebo Stop() na FMOD instanci.

FMODEvents – centrální registr eventů

Propojení Unity a FMOD na úrovni kódu zajišťuje singleton `FMODEvents.cs`, který drží téměř všechny `EventReference` na jednom místě:

```
1 public class FMODEvents : MonoBehaviour
2 {
3     [field: Header("Music")]
4     [field: SerializeField] public EventReference music { get; private
        set; }
5
6     [field: Header("UI")]
7     [field: SerializeField] public EventReference buttonClick { get;
        private set; }
8     [field: SerializeField] public EventReference pause { get; private
        set; }
9
10    [field: Header("Diegetic/Player")]
11    [field: SerializeField] public EventReference footsteps { get;
        private set; }
12    [field: SerializeField] public EventReference jump { get; private
        set; }
13    [field: SerializeField] public EventReference dash { get; private
        set; }
14
15    public static FMODEvents instance { get; private set; }
16 }
```

Kód 5.17: FMODEvents – centrální registr téměř všech FMOD eventů

Jednotlivé FMOD eventy jsou v tomto skriptu definovány jako serializovatelná pole, která lze nastavit v Unity Inspectoru. Tím se odděluje herní logika od konkrétního zvukového obsahu – změna zvuku nevyžaduje úpravu kódu, pouze přebindování `EventReference` v editoru.

Architektura přehrávání zvuků

V nejnovější verzi projektu existují dva hlavní, odlišné přístupy k přehrávání zvuků, které odrážejí různou povahu zvukových zdrojů.

PlayOneShot – jednorázové zvuky

První přístup využívá `PlayOneShot()` – metodu, která vytvoří dočasnou FMOD instanci, přehraje zvuk na zadané pozici a instanci ihned uvolní. Tento model je ideální pro krátké, jednorázové zvuky, u kterých nepotřebujeme sledovat jejich životní cyklus. Příkladem je smrt hráče, kde `PlayerDeath.cs` volá:

```

1 AudioManager.instance.PlayOneShot(FMODEvents.instance.death, this.
  transform.position);

```

Kód 5.18: PlayOneShot – jednorázový zvuk

StudioEventEmitter – looping objekty

Druhý přístup využívá StudioEventEmitter komponentu připojenou přímo k GameObjectu. Tento model je vhodný pro zvuky, které jsou vázány na životní cyklus objektu nebo na jeho herní stav. Pro *looping* objekty, jako je valící se *Ball*, je na tomto GameObjectu v prefabu připojena komponenta StudioEventEmitter s nastavením EventPlayTrigger = ObjectStart a EventStopTrigger = ObjectDestroy. FMOD event se tak spustí při vytvoření objektu a hraje v nekonečné smyčce, dokud objekt není zničen (např. při dopadu do destrukční zóny), pak se event ukončí s fade-outem.

One-shot objekty s vlastním emitorem

Pro *one-shot* objekty, které mají svůj vlastní event emitore, existují dva přístupy. Prvním je vázání na ObjectDestroy – když je GameObject zničen (například při sebrání collectible), FMOD event se přehraje. Typickým příkladem je Collectible (CF – Conversation Fragment), kde druhý StudioEventEmitter na objektu přehrává zvuk sebrání:

```

1 // Emitter 1 -- idle/ambient zvuk
2 StudioEventEmitter:
3   EventReference: event:/SFX/Diegetic/Environment/CF/CFIdle
4   EventPlayTrigger: ObjectStart
5   EventStopTrigger: ObjectDestroy
6
7 // Emitter 2 -- zvuk řpi sebrání
8 StudioEventEmitter:
9   EventReference: event:/SFX/Diegetic/Environment/CF/CFCollect
10  EventPlayTrigger: ObjectDestroy
11  EventStopTrigger: None

```

Kód 5.19: Collectible prefab – dva FMOD emitory

Na objektu CF jsou tedy dva StudioEventEmitter komponenty – první se spustí při vytvoření objektu a vydává ambientní zvuk, druhý se spustí až při zničení objektu (sebrání hráčem). Druhý přístup pro one-shot objekty spočívá ve využití RuntimeManager.PlayOneShot() přímo v kódu, což je vhodné pro situace, kdy chceme mít plnou kontrolu nad timingem přehrávání.

Vazba zvuků na animace

Zvuky je možné navázat přímo na animace pomocí Unity *Animation Events*. Tato technika umožňuje, aby v konkrétním snímku animace byla zavolána libovolná metoda na libovolném skriptu připojeném k GameObjectu. Příkladem jsou kroky hráče – animace player_run.anim obsahuje dva Animation Events:

```

1 m_Events:
2   - time: 0.1
3     functionName: PlayFootsteps
4     objectReferenceParameter: {fileID: 0}
5   - time: 0.33333334
6     functionName: PlayFootsteps
7     objectReferenceParameter: {fileID: 0}

```

Kód 5.20: Animation Event v player_run.anim – PlayFootsteps

Tyto events volají metodu PlayFootsteps() na skriptu PlayerMovementAudio.cs, který je připojen k objektu hráče:

```

1 public class PlayerMovementAudio : MonoBehaviour
2 {
3     public void PlayFootsteps()
4     {
5         AudioManager.instance.PlayOneShot(FMODEvents.instance.footsteps
6             , transform.position);
7     }
8     public void PlayJump()
9     {
10        AudioManager.instance.PlayOneShot(FMODEvents.instance.jump,
11            transform.position);
12    }

```

Kód 5.21: PlayerMovementAudio – obsluha footsteps a jump

Metoda PlayFootsteps() je volána ve dvou snímcích běžící animace (na 0.1 s a 0.333 s), což odpovídá přirozenému rytmu běhu. Podobně animace player_jump.anim obsahuje Animation Event na snímku 0s, který volá PlayJump(). Výhodou tohoto přístupu je dokonalá synchronizace zvuku s animací bez nutnosti sledovat stav animace v kódu.

Hudba a Ambience

Hudba a ambience jsou v FMOD řešeny jako kontinuální eventy, které se nespouštějí přes PlayOneShot(), ale přes správu *EventInstance*. Nový AudioManager.cs vytváří při startu dvě dlouhožijící instance – jednu pro hudbu a jednu pro ambience:

```

1 public class AudioManager : MonoBehaviour
2 {
3     private EventInstance musicEventInstance;
4     private EventInstance ambienceEventInstance;

```

```

5     private List<EventInstance> eventInstances;
6
7     private void Start()
8     {
9         InitializeAmbience(FMODEvents.instance.ambience);
10        InitializeMusic(FMODEvents.instance.music);
11    }
12
13    private void InitializeMusic(EventReference musicEventReference)
14    {
15        musicEventInstance = CreateEventInstance(musicEventReference);
16        musicEventInstance.start();
17    }
18 }

```

Kód 5.22: AudioManager – správa hudby a ambience pomocí EventInstance

Pro přepínání hudby a ambience mezi scénami využívám systémy MusicChangeTrigger a AmbienceChangeTrigger, které reagují na událost SceneManager.sceneLoaded. Každá scéna má definovanou odpovídající hodnotu parametru:

```

1 public enum SceneMusic
2 {
3     MAIN_MENU = 0,
4     LEVEL_1 = 3,
5     LEVEL_2 = 7
6 }

```

Kód 5.23: SceneMusic enum – mapování scén na FMOD parametry

V FMOD Studiu je na hudebním eventu vytvořen diskretní Parameter s názvem SceneMusic. Hudební event obsahuje pro každou hodnotu parametru navázanou jinou hudební stopu – při změně parametru na hodnotu 3 se crossfadem začne přehrávat hudba pro Level 1, při změně na 0 hudba pro MainMenu. Unity skript pak volá:

```

1 public void SetSceneMusic(SceneMusic scene)
2 {
3     if (!musicEventInstance.isValid()) return;
4
5     float targetValue = (float)scene;
6     musicEventInstance.setParameterByName("SceneMusic", targetValue);
7 }

```

Kód 5.24: MusicChangeTrigger – změna hudby při načtení scény

Systém AmbienceChangeTrigger funguje identicky, pouze s enumem SceneAmbience a FMOD parametrem SceneAmbience. Toto řešení má výhodu v tom, že oba zvukové systémy jsou perzistentní

mezi scénami a přechody jsou plynulé, protože crossfading řeší FMOD interně na základě nastavení parametru. Technicky je efekt plynulého přechodu realizován pomocí automatizace hlasitosti – v FMOD Studiu je v *Parameter Sheet* každé hudební stopy nastaven AHDSR (Attack-Hold-Decay-Sustain-Release) envelope, konkrétně modul *Volume* s automatizační křivkou, která definuje hlasitostní křivku pro každou hodnotu parametru *SceneMusic* nebo *SceneAmbience*. Když skript změní hodnotu parametru, FMOD interpoluje mezi těmito automatizačními body a výsledkem je hladký crossfade mezi jednotlivými stopami bez nutnosti jakéhokoliv kódu navíc.

5.4.2 Tvorba v FL Studio

5.4.3 Kategorizace zvuků

5.4.4 Hudba

5.5 Post-processing hry

- ukázka Global Volume post processing prvku

6 Závěr

Seznam obrázků

5.1	Rozhraní Unity Input System	18
5.2	Hierarchie scény	21
5.3	GameObject hráče ve scéně	21
5.4	Rozhraní UI Toolkit	23
5.5	Nastavení hlasitosti	25
5.6	HUD ve hře – levý horní roh: počet střel, pravý horní roh: počet CFs	26
5.7	Bližší parallax vrstva (parallaxEffectX = 0.45)	27
5.8	Vzdálenější parallax vrstva (parallaxEffectX = 0.7)	27
5.9	Světlušky v korunách stromů ve scéně	30
5.10	Struktura Unity AudioMixeru – hierarchie kanálů	32